

① 活性边表, 多边形填充

1 Edge Bucket

⇔ 边的数据

结构:

y_{max} : 该边的最大 y 坐标

y_{min} : 该边的最小 y 坐标

x : 扫描线当前的 x 坐标

$sign$: 斜率的符号

dx : $|x_0 - x_{end}|$

dy : $|y_0 - y_{end}|$

2. Edge Table ET

边表包含了构成图形的所有边;

注意 (1) 顶点从左往右排列 (x_{min})

(2) 边以 y_{min} 递增的顺序排列

(3) Active List 处理完某边后, 要马上从 ET 中删除

3. Active List

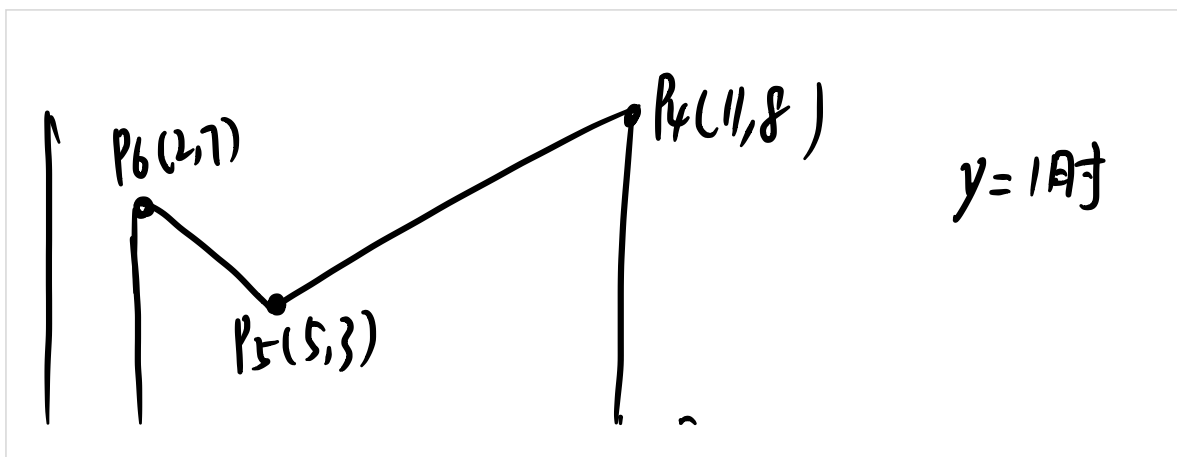
1° 当扫描线的 y 坐标与 ET 中某些边的 y_{min} 相等时,

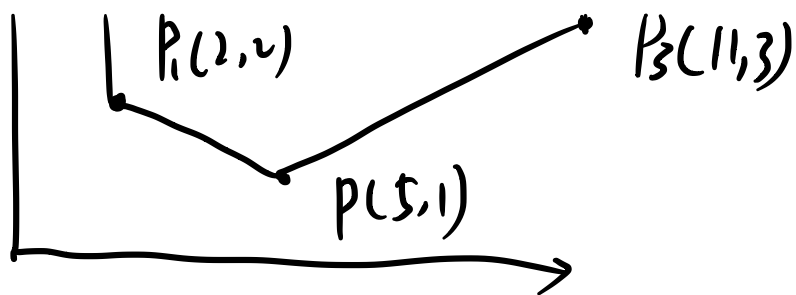
应把那些边加入 AL

2° 当 y 坐标 = AL 中某些边的 y_{max} 时, 应把那些边从 AL 中弹出

3° AL 中的边以 x 递增的顺序排列。

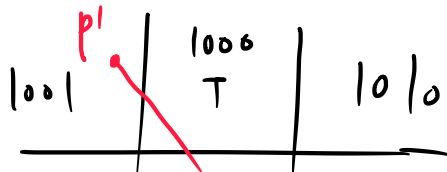
Example

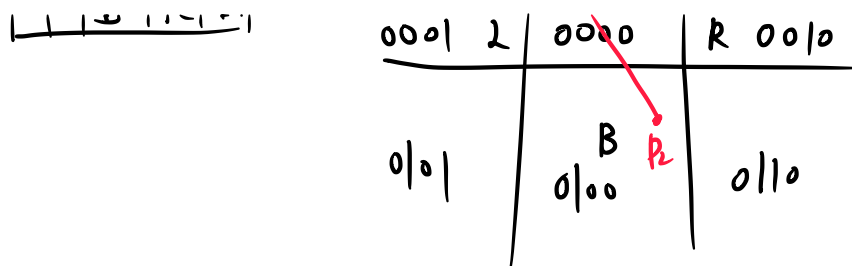




② 直线裁剪 Cohen-Sutherland

1° 首先我们要了解Cohen-Sutherland算法的核心是
区间分类.





2° 根据直线两个端点的二进制数值初步判断直线与矩形的位关系

(1): 若两端点二进制值 OR 后为 0, 则必在矩形内

(2): 若两端点二进制值 XOR 后为 1, 则必在矩形外

(3): 若不符合以上两种情况, 则直线可能与矩形相交, 需进一步探讨.

按左 → 右 → 下 → 上的方位依次检测.

检测方法: 若对应方向位异或结果为 1,

则说明相交,

以直线 P1P2 为例.

$P_1: (1001) \quad P_2: (0100)$

左侧 $1 \wedge 0 = 1$, 相交, 更新方法:
$$\begin{cases} y_{new} = m(x_L - x_0) + y_0 \\ x_{new} = x_L \end{cases}$$

右侧 $0 \wedge 0 = 0$ 不相交

下侧 $0 \wedge 1 = 1$ 相交, 更新
$$\begin{cases} x_{new} = \frac{1}{m}(y_B - y_0) + x_0 \\ y_{new} = y_L \end{cases}$$

上侧 $1 \wedge 0 = 1$ 相交, 更新

$$\begin{cases} x_{new} = \frac{1}{m}(y_T - y_0) + x_0 \\ y_{new} = y_T \end{cases}$$

试题1.

贝塞尔的起点、中点、终点、
端点导数.

在平面上特征多边形顶点 $p_0(0,0)$, $p_1(1,1)$ $p_2(2,-1)$

确定一条二次贝塞尔曲线 $C(t)$, $t \in [0,1]$

a) 求该曲线的起点、中点、终点坐标

b) 求 $C'(0)$ 和 $C'(1)$

解法.

$$\begin{aligned}\text{解(1): } C(t) &= \sum_{i=0}^n B_{2,i}(t) \cdot p_i = \begin{pmatrix} 2 \\ 0 \end{pmatrix} (1-t)^2 p_0 + \begin{pmatrix} 2 \\ 1 \end{pmatrix} t(1-t) p_1 + \begin{pmatrix} 2 \\ 2 \end{pmatrix} t^2 p_2 \\ &= (1-t)^2 p_0 + 2t(1-t) p_1 + t^2 p_2 \\ &= 2 \cdot (t-t^2) \cdot \begin{Bmatrix} 1 \\ 1 \end{Bmatrix} + t^2 \cdot \begin{Bmatrix} 2 \\ -1 \end{Bmatrix}\end{aligned}$$

$$\begin{aligned}\text{当 } t=0 \text{ 时, 为起点 } [0, 0] \\ t=\frac{1}{2} \text{ 时, 中点 } [1, \frac{1}{4}] \\ t=1 \text{ 时, 终点 } [2, -1]\end{aligned} \quad = \begin{Bmatrix} 2t \\ 2t-3t^2 \end{Bmatrix}$$

$$\text{解(2): } C'(t) = (2t-2)p_0 + 2 \cdot (1-2t)p_1 + 2tp_2$$

$$C'(0) = -2p_0 + 2p_1 = \begin{Bmatrix} 2 \\ 2 \end{Bmatrix}$$

$$C'(1) = -2p_1 + 2p_2 = \begin{Bmatrix} 4-2 \\ -2-2 \end{Bmatrix} = \begin{Bmatrix} 2 \\ -4 \end{Bmatrix}$$

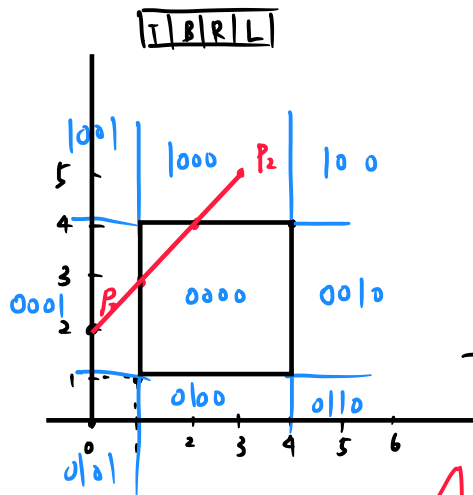
试题二

直线裁剪算法

用编码裁剪线段 $P_1(0, 2)$, $P_2(3, 5)$, 裁剪窗口左下角为 $(1, 1)$, 右上角为 $(4, 4)$

- 写出线段端点的编码
- 裁剪后窗口内的表示

解



① $P_1: 0001 \quad P_2: 1000$

② 由于 $P_1 | P_2 \neq 0$

$P_1 \& P_2 = 0$

故没有办法确定 P_1 的位置，

须进一步判断

$$\begin{array}{r} 0001 \\ \wedge 1000 \\ \hline 1001 \\ T \quad L \end{array}$$

由异或结果可知，直线在
矩形左侧与上侧有交点，

先求左侧：

$$P_1' \begin{cases} y_{new} = (x_L - x_0) \cdot k + y_0 = 3 \\ x_{new} = x_L = 1 \quad [3, 1] \end{cases}$$



$\Delta y = \Delta x \cdot \tan \theta$

再求上侧：

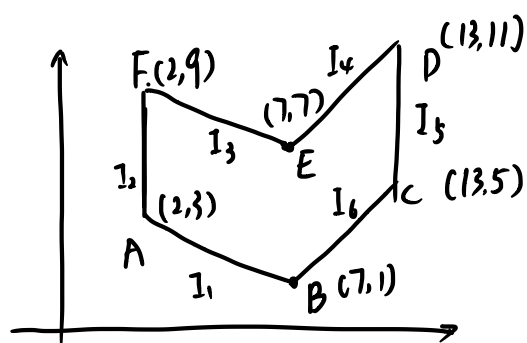
$$P_2' \begin{cases} y_{new} = y_T = 4 \\ x_{new} = (y_{new} - y_0) \cdot \frac{1}{k} + x_0 \\ = (4 - 2) \cdot \frac{1}{k} + 0 = 2 \end{cases}$$

$P_2' [4, 2]$

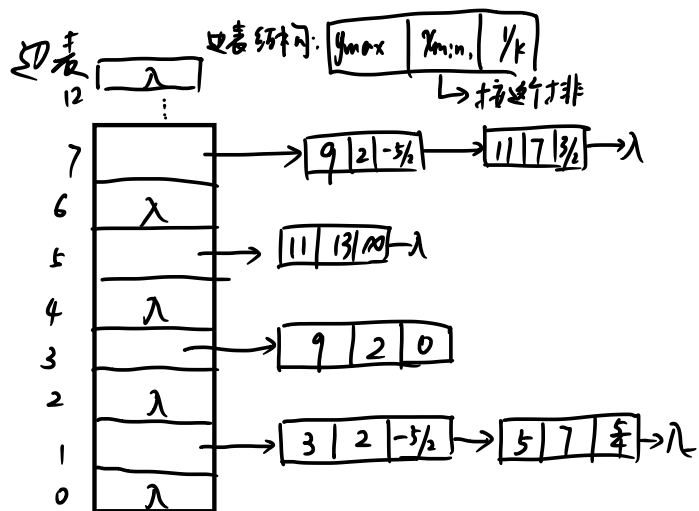
故新直线为 $P_1' [3, 1] \quad P_2' [4, 2]$

试题三、活性边表、边表

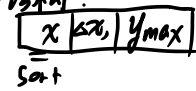
写出下图所示多边形的边表和扫描线 $y=4$ 的有效表



解:

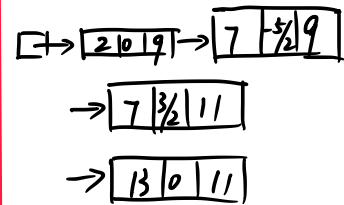


活桩边表结构:



$$\frac{\Delta y}{\Delta x} \tan \theta = \frac{\Delta y}{\Delta x}; \Delta x = \Delta y \frac{1}{\tan \theta} = 1 \times \frac{1}{k}$$

$y=7$ 时的边表为 -



简答题

1、计算机图形学、图像处理、模式识别几个关系和区别：

图像处理、模式识别与计算机图形学是计算机应用领域发展的三个分支学科，它们之间有一定的关系和区别，它们的共同之处就是计算机所处理的信息都是与图有关的信息。

它们本质上是不同的：

计算机图形学是研究根据给定的描述,用计算机生成相应的图形、图像,且所生成的图形、图像可以显示屏幕上、硬拷贝输出或作为数据集存在计算机中的学科。计算机图形学研究的是从数据描述到图形生成的过程。例如计算机动画制作。

图形处理是利用计算机对原来存在物体的映像进行分析处理,然后再现图像。例如工业中的射线探伤。

模式识别是指计算机对图形信息进行识别和分析描述,是从图形(图像)到描述的表达过程。例如邮件分拣设备扫描信件上手写的邮政编码,并将编码用图像复原成数字。

2、图形处理和图像处理有何区别？有何关联？

2、图形处理和图像处理有何区别？它们之间有何关联？

答：（1）概念不同，图形是指由外部轮廓线条构成的矢量图。即由计算机绘制的直线、圆、矩形、曲线、图表等；而图像是由扫描仪、摄像机等输入设备捕捉实际的画面产生的数字图像，是由像素点阵构成的位图。（2）数据描述不同，图形：用一组指令集合来描述图形的内容，如描述构成该图的各种图元位置维数、形状等。描述对象可任意缩放不会失真。图像：用数字任意描述像素点、强度和颜色。描述信息文件存储量较大，所描述对象在缩放过程中会损失细节或产生锯齿。（3）屏幕显示不同，图形：使用专门软件将描述图形的指令转换成屏幕上的形状和颜色。图像：是将对象以一定的分辨率分辨以后将每个点的信息以数字化方式呈现，可直接快速在屏幕上显示。

图形与图像制成的图样，通称图案。

答: (1)概念不同, 图形是指由外部轮廓线条构成的矢量图。即由计算机绘制的直线、圆、矩形、曲线、图表等;

而图像是由扫描仪、摄像机等输入设备捕捉实际的画面产生的数字图像, 是由像素点阵构成的位图,

(2)数据描述不同:

图形:用一组指令集合来描述图形的内容, 如描述构成该图的各种图元位置维数、形状等。描述对象可任意缩放不会失真。

图像:用数字任意描述像素点、强度和颜色。描述信息文件存储量较大, 所描述对象在缩放过程中会损失细节产生锯齿。

(3) 屏幕显示不同:

图形:使用专门软件将描述图形的指令转换成屏幕上的形状和颜色。

图像:是将对象以一定的分辨率分辨以后将每个点的信息以数字化方式呈现, 可直接快速在屏幕上显示。。

图形与图像制成的图样, 通称图案。



3、什么是走样（失真）、反走样，如果出现了走样，如何反走样，有哪些操作可以反走样。

走样指的是用离散量表示连续量引起的失真。为了提高图形的显示质量。需要减少或消除因走样带来的阶梯形或闪烁效果,用于减少或消除这种效果的方法称为反走样。其方法是1前滤波,以较高的分辨率显示对象;2后滤波,即加权区域取样,在高于显示分辨率的较高分辨率下用点取样方法计算,然后对几个像素的属性进行平均得到较低分辨率下的像素属性。

3、什么是反走样？并简要阐述1种你熟悉的反走样技术。

走样：

在光栅化时，将浮点坐标的几何图形离散化为整数的像素位置，因此光栅算法生成的图元显示具有锯齿形或阶梯状外观，这种由于低频取样（不充分取样）而造成的信息失真称为走样，也就是指的是用离散量表示连续量引起的失真。比如，直线或曲线往往呈现锯齿状，细小物体在图上显示不出来等。这种现象就是图形的走样

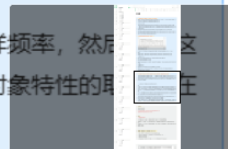
反走样：为了提高图形的显示质量。需要减少或消除因走样带来的阶梯形或闪烁效果,用于减少或消除这种效果的方法，通过使用校正不充分取样的过程来改善所显示的光栅图元的外观。

其方法是1前滤波,以较高的分辨率显示对象;2后滤波,即加权区域取样,在高于显示分辨率的较高分辨率下用点取样方法计算,然后对几个像素的属性进行平均得到较低分辨率下的像素属性。

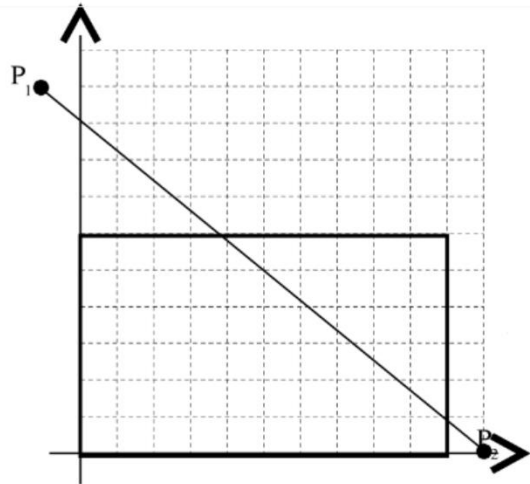
一种简单、直接的反走样方法，就是把屏幕看成由比实际更细的网格所覆盖，从而增加取样频率，然后在这

一种更细的网格，使用取样点来确定每个屏幕像素的合适亮度等级。这种在高分辨率下对于对象特性的取

在较低分辨率上显示其结果的技术称为过取样/多重采样抗锯齿。

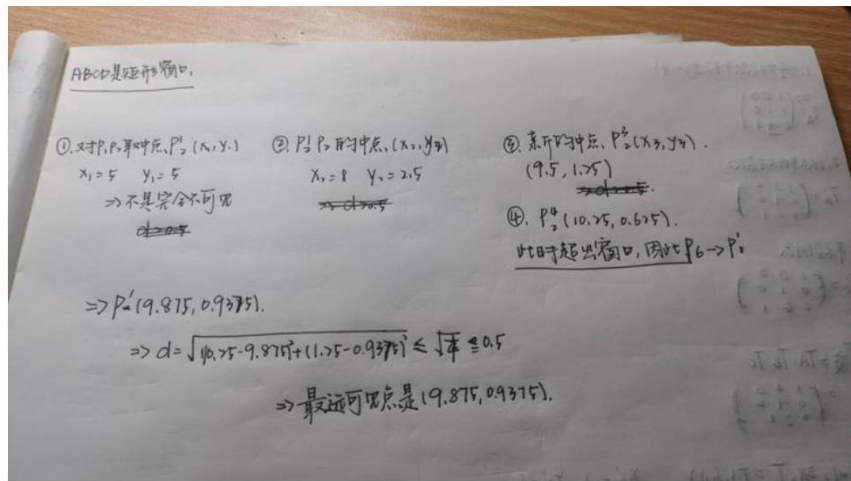


图中ABCD为矩形窗口，P1、P2为待裁剪线段。试用中点分割法求出P1的最远可见点，当线段长度 ≤ 0.5 时算法结束。
已知窗口及线段的坐标分别为A(0, 0), B(0, 6), C(10, 6), D(10, 0), P1(-1, 10), P2(11, 0)



25

我的答案:



设有控制顶点为P0(0,0), P1(48,96), P2(120,120), P3(216,72)的三次Bezier曲线P(t)，试计算P(0.4)的(x,y)坐标，并写出((x(t),y(t))的多项式表示。

我的答案:

25 分

$$\begin{aligned}
 4. \quad p(t) &= (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3 \\
 p(0.4) &= (0.6)^3 P_0 + 1.2 \cdot 0.6^2 \cdot P_1 + 3 \cdot 0.4 \cdot 0.6 \cdot P_2 + 0.4^3 P_3 \\
 &= (69.12, 80.64)
 \end{aligned}$$

4. (简答题)

设投影参考点为 $(0, 0, d)$ ，投影面为 xoy 平面，请推导投影变换矩阵。

25 分

我的答案:

投影参考点为 $(0, 0, d)$ ，投影面是 xoy 平面，请推导投影变换矩阵。

$\Rightarrow$ 首先，由于投影参考点是 $(0, 0, d)$ ， $\Rightarrow$ 由于投影面是 xoy 平面，
 根据 $PP' = \begin{cases} X' = (X-0)u \\ Y' = (Y-0)u \\ Z' = (Z-d)u + d \end{cases}$ 于是 $Z'=0, \Rightarrow zu + d = 0 \Rightarrow u = \frac{d}{d-Z}$

由 $\frac{d}{d-Z}$ 是比例因子，
 $\Rightarrow$ 于是 $\frac{X'}{d-Z} = \frac{X}{d-Z}$ 即 $X' = \frac{X}{d-Z} \cdot d$

投影变换矩阵是

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{d} & 1 \end{pmatrix}$$

正确答案:

1. (简答题) 用中点画线法扫描转换从点 $(1, 0)$ 到 $(4, 7)$ 的直线段，并给出每一步的判别量。

25 分

我的答案:

$(1, 0)$ 到 $(4, 7)$ ，
 $k = \frac{7-0}{4-1} = \frac{7}{3}$ $|k| > 1$ ， $a = y_1 - y_0 = -3$
 因此要写坐标反序表： $b = x_1 - x_0 = 3$
 得 $k = \frac{3}{7} \Rightarrow$ 从 $(0, 1)$ 到 $(7, 4)$ 。
 $\Rightarrow -3x + 7y + 7 = 0$
 由于只用 d 的变异性判断，
 用 d 来代表 d ，
 且有 $d_0 = 2a + b = 1$ ， $d_1 = 2a = -6$ ， $d_2 = 2a + b = 8$
 得：

i	x_i	y_i	d_i
1	0	1	1
2	1	1	-5
3	2	2	3
4	3	2	-3
5	4	3	5
6	5	3	-1
7	6	4	7
8	7	4	1

最后将 y, x 倒过来即可。

经过 x, y 翻转后得：

$(1, 0)$
$(1, 1)$
$(2, 2)$
$(2, 3)$
$(3, 4)$
$(3, 5)$
$(4, 6)$
$(4, 7)$

2、要求利用某种图形函数库（例如 OpenGL、GLUT）所提供的函数接口实现在窗口内移动一个五角星的完整程序。具体要求实现键盘交互，按 U 或 u 键上移，按 D 或 d 键下移，按 L 或 l 键矩形左移，按 R 或 r 键右移。设定每次位移的步长为 10 个像素单位，窗口大小 500*500，五角星边长为 50 个像素点。要求：不用写出框架性的代码，可直接写出任意直线生成模块、五角星的生成、键盘交互模块等你认为比较关键模块的代码即可，代码中要有适当注释。

要实现在窗口内移动五角星，您可以使用 OpenGL 和 GLUT 来创建窗口和绘制图形，然后使用键盘回调函数来实现交互。以下是实现该程序的基本步骤：

导入必要的头文件：

c++

Copy code

```
#include <GL/glut.h>
```

```
#include <cmath>
```

定义窗口大小和五角星的边长：

c++

Copy code

```
int window_width = 500;
```

```
int window_height = 500;
```

```
int star_size = 50;
```

定义五角星的坐标：

c++

Copy code

```
float star_x = window_width / 2;
```

```
float star_y = window_height / 2;
```

绘制五角星的函数：

c++

Copy code

```
void drawStar(float x, float y, float size) {
```

```
    float angle = 4 * atan(1) / 5; // 计算五角星内角的一半
```

```
    float radius = size / (2 * sin(angle)); // 计算五角星外接圆半径
```

```
    glBegin(GL_POLYGON);
```

```
    for (int i = 0; i < 5; i++) {
```

```
        glVertex2f(x + radius * sin(i * 2 * angle), y + radius * cos(i * 2 * angle)); // 绘制五
```

```
    角星的五个顶点
```

```
    }
```

```
    glEnd();
```

```
}
```

在 display 回调函数中绘制五角星：

c++

Copy code

```
void display() {
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glColor3f(1.0f, 1.0f, 0.0f); // 设置五角星颜色为黄色
```

```
    drawStar(star_x, star_y, star_size); // 绘制五角星
```

```
    glutSwapBuffers();
```

```
}
```

实现键盘回调函数，以便通过按键来移动五角星：

c++

Copy code

```
void keyboard(unsigned char key, int x, int y) {  
    switch (key) {  
        case 'u': // 上移  
        case 'U':  
            star_y += 10;  
            break;  
        case 'd': // 下移  
        case 'D':  
            star_y -= 10;  
            break;  
        case 'l': // 左移  
        case 'L':  
            star_x -= 10;  
            break;  
        case 'r': // 右移  
        case 'R':  
            star_x += 10;  
            break;  
        default:  
            break;  
    }  
    glutPostRedisplay(); // 标记窗口需要重绘  
}
```

在主函数中注册回调函数并启动事件循环：

c++

Copy code

```
int main(int argc, char** argv) {  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);  
    glutInitWindowSize(window_width, window_height);  
    glutCreateWindow("Moving Star");  
    glutDisplayFunc(display);  
    glutKeyboardFunc(keyboard);  
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f); // 设置窗口背景颜色为黑色  
    glutMainLoop();  
    return 0  
}
```

任意直线生成呢

如果您需要在 OpenGL 中绘制直线，可以使用 `glBegin` 和 `glEnd` 函数以及 `GL_LINES` 参数来绘制直线段。具体步骤如下：

在 `display` 回调函数中添加绘制直线的代码，例如：

C++

Copy code

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 0.0f); // 设置颜色为黄色
    drawStar(star_x, star_y, star_size); // 绘制五角星
    glColor3f(0.0f, 1.0f, 0.0f); // 设置颜色为绿色
    glBegin(GL_LINES);
    glVertex2f(100.0f, 100.0f); // 设置直线的起点
    glVertex2f(400.0f, 400.0f); // 设置直线的终点
    glEnd();
    glutSwapBuffers();
}
```

在主函数中运行程序，可以看到在窗口中绘制了一条从(100,100)到(400,400)的绿色直线。如果您需要绘制其他类型的直线，例如直线段、线带或线环等，可以根据需要在 `glBegin` 和 `glEnd` 函数中使用不同的参数，例如 `GL_LINE_STRIP`、`GL_LINE_LOOP` 等。如果您需要绘制更复杂的图形，可以使用 OpenGL 提供的其他函数和参数进行绘制。

3、现有图形显示设备的扫描模式一般采用哪种模式？为在屏幕上输出一条任意斜率的直线，一般受到哪些因素的影响？

答：一般采用隔行光栅扫描显示的模式；

在光栅图形显示器上输出一条任意斜率的直线，主要受到以下因素的影响：

- （1）光栅图形显示器的分辨率；
- （2）线宽、线型；
- （3）直线的扫描转换的算法

4、写出 2 维条件下具有 5 个控制顶点的 Bezier 曲线所有混合函数的定义，给出每个混合函数在参数为何值取得最大值和最小值（提示：参数的取值范围为 0 到 1 之间）。

答：

(1)

$$B_0(t) = (1-t)^4, t \in [0, 1]$$

$$B_1(t) = 4t(1-t)^3, t \in [0, 1]$$

$$B_2(t) = 6t^2(1-t)^2, t \in [0, 1]$$

$$B_3(t) = 4t^3(1-t), t \in [0, 1]$$

$$B_4(t) = t^4$$

(2) ~~$B_0(t) = (1-t)^4$~~ $B_0(t)$ 在 $t=0$ 取最大值, $t=1$ 取最小值

$B_1(t)$ 在 $t=\frac{1}{4}$ 取最大值, $t=0$ 取最小值

$B_2(t)$ 在 $t=\frac{1}{2}$ 取最大值, $t=0$ 取最小值

$B_3(t)$ 在 $t=\frac{3}{4}$ 取最大值, $t=0$ 取最小值

$B_4(t)$ 在 $t=1$ 取最大值, $t=0$ 取最小值

1、假设有一单位球体 $S = \{(x, y, z) \mid x^2 + y^2 + z^2 = 1\}$ 。在该球体上连续执行平移和尺度变换，其中，在平移变换中 X、Y 和 Z 坐标轴上的偏移量分别为 2、5 和 6 个单位；在尺度变换中 X、Y 和 Z 坐标轴上的比例系数分别为 1、2 和 2。请完成以下工作：

(1) 请写出变换矩阵；

(2) 假定该单位球面上表面上某点（其坐标为 $(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})$ ），计算经过上述图形变换后，该点在新位置上的坐标值；

(3) 计算点 a 在新位置处的法向量。

答：

(1) 平移 $T = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & 6 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

缩放 $S = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

总变换矩阵 $M = S \cdot T = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 2 & 0 & 10 \\ 0 & 0 & 2 & 12 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

(2) $a = (\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}})^T$
 $a' = M \cdot a = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 2 & 0 & 10 \\ 0 & 0 & 2 & 12 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{3}} + 2 \\ \frac{2}{\sqrt{3}} + 10 \\ \frac{2}{\sqrt{3}} + 12 \\ 1 \end{pmatrix}$

(3) 任意点 (x, y, z) 经变换后为 (x', y', z')

满足 $\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 2 & 0 & 10 \\ 0 & 0 & 2 & 12 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$

$\Rightarrow \begin{cases} x' = x + 2 \\ y' = 2y + 10 \\ z' = 2z + 12 \end{cases} \Rightarrow \begin{cases} x = x' - 2 \\ y = (y' - 10)/2 \\ z = (z' - 12)/2 \end{cases}$

代入 $x^2 + y^2 + z^2 = 1 \Rightarrow (x' - 2)^2 + \frac{1}{4}(y' - 10)^2 + \frac{1}{4}(z' - 12)^2 = 1$

求偏导： $\begin{cases} 2(x' - 2) \\ \frac{1}{2}(y' - 10) \\ \frac{1}{2}(z' - 12) \end{cases}$

得 $a' = \begin{pmatrix} \frac{2}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{pmatrix}$ 代入上式 (求法向量)

$\begin{pmatrix} 2/\sqrt{3} \\ 1/\sqrt{3} \\ 1/\sqrt{3} \end{pmatrix}$

GAMES101 Transformation



Wool
混沌

关注他

Transformation

- Why study transformation
- 2D transformation: rotation, scale, shear
- Homogeneous coordinates
- Composing transforms
- 3D transformations
- Viewing transformation

一、Why transformation

- Modeling (animation transformation, camera moving)
- Viewing (Point of observation 3D world -> 2D image projection)

二、2D transformations

1. Scale

(1) Scale Transform

$$\begin{aligned}x' &= sx \\ y' &= sy\end{aligned}$$

(2) Scale Matrix

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

(3) Scale Non-Uniform

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2. Reflection Matrix

(1) Horizontal reflection

$$\begin{aligned}x' &= -x \\ y' &= y\end{aligned}$$

(2) Matrix

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

3. Shear Matrix

- Horizontal shift is 0 at $y = 0$
- Horizontal shift is a at $y = 1$

赞同

添加评论

分享

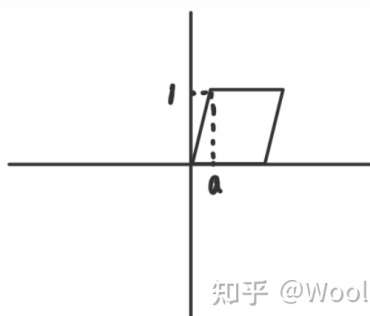
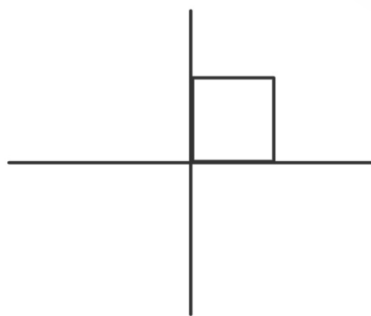
喜欢

收藏

申请转载

...

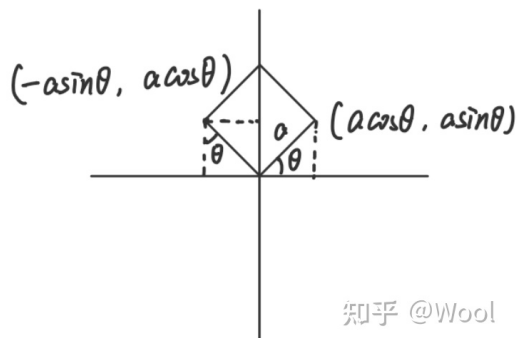
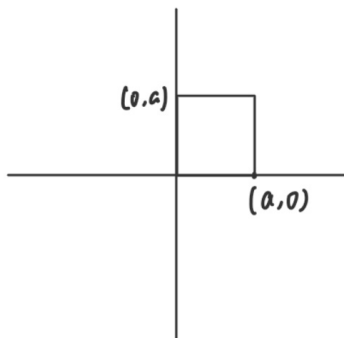
- Vertical shift is always 0



知乎 @Wool

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

4. Rotate (about the origin(0, 0), CCW by default)



知乎 @Wool

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

$$\begin{pmatrix} a \cos \theta \\ a \sin \theta \end{pmatrix} = \begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} a \\ 0 \end{pmatrix} \quad \begin{matrix} A = \cos \theta \\ C = \sin \theta \end{matrix}$$

$$\begin{pmatrix} -a \sin \theta \\ a \cos \theta \end{pmatrix} = \begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} 0 \\ a \end{pmatrix} \quad \begin{matrix} B = -\sin \theta \\ D = \cos \theta \end{matrix}$$

$$\therefore R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Inverse Rotation

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

$$R_{-\theta} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} = R_\theta^T$$

$$R_{-\theta} = R_\theta^{-1} \text{ (By definition)}$$

$$\therefore R_{-\theta} = R_\theta^T \text{ (Orthogonal Matrix)}$$

5. Linear Transforms = Matrix

▲ 赞同 ▼

● 添加评论

↗ 分享

♥ 喜欢

★ 收藏

📄 申请转载

...

$$x' = ax + by$$

$$y' = cx + dy$$

$$x' = Mx$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

三、Homogeneous coordinates

1. Why Homogeneous Coordinates

(1) Translation

$$x' = x + t_x$$

$$y' = y + t_y$$

Cannot be represented in matrix form

It's NOT linear transform!

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

But we don't want translation to be a special case.

Find a unified way to represent all transformations.

But what's the cost?

2. Solution:

Homogeneous Coordinates

Add a third coordinate (w-coordinate)

$$2D \text{ points} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$2D \text{ vectors} = \begin{pmatrix} x \\ y \\ 0 \end{pmatrix} \quad \text{向量平移不变性 加 0 保护向量不变}$$

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

Valid operation if w-coordinate of result is 1 or 0

- vector + vector = vector
- point - point = vector
- point + vector = point
- point + point = ??

In homogeneous coordinates:

$$\begin{pmatrix} x \\ y \\ w \end{pmatrix} \text{ is the 2D point } \begin{pmatrix} x/w \\ y/w \\ w \end{pmatrix}, w \neq 0$$

3. Affine Transformations

▲ 赞同 ▼

● 添加评论

↗ 分享

♥ 喜欢

★ 收藏

📄 申请转载

...

Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

Using homogeneous coordinates

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

4. 2D Transformations

(1) Scale

$$S(S_x, S_y) = \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

(2) Rotation

$$R(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

(3) Translation

$$T(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

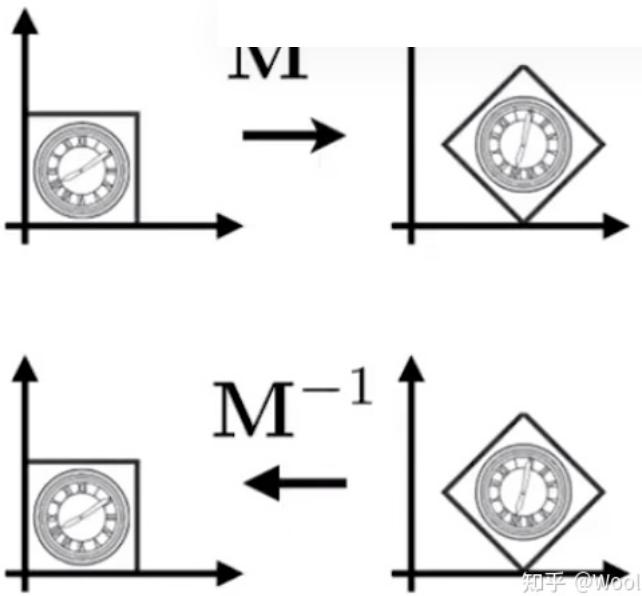
In Affine Transformation, 0 0 1

四、Composite Transform

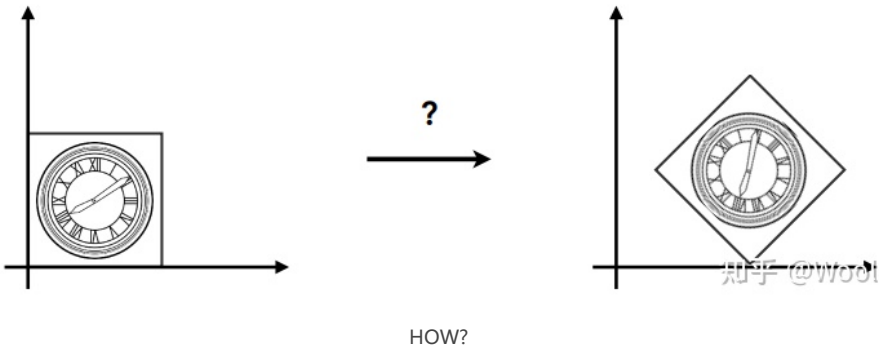
1. Inverse Transform

$$M^{-1}$$

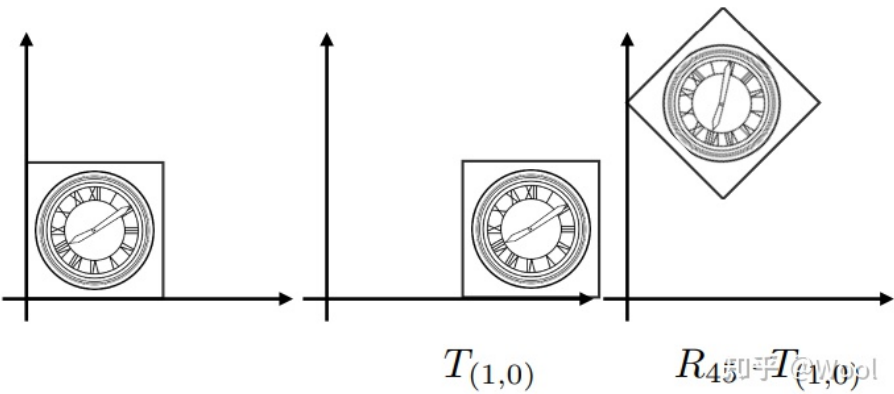
M^{-1} is the inverse of transform M in both a matrix and geometric sense.



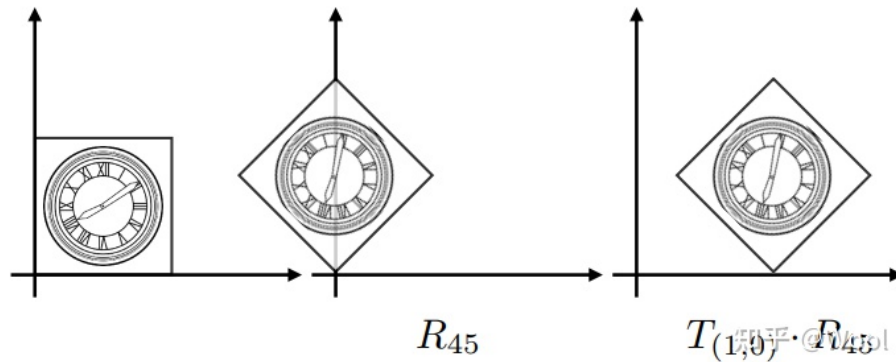
2. Transform Order Matters



Translate Then Rotate?



Rotate Then Tra



Matrix multiplication is not commutative

$$R_{45} \cdot T_{(1,0)} \neq T_{(1,0)} \cdot R_{45}$$

Note that matrices are applied right to left

$$T_{(1,0)} \cdot R_{45} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \sin 45^\circ & -\sin 45^\circ & 0 \\ \cos 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

3. Composing Transforms

Sequence of affine transforms $A_1, A_2, A_3 \dots$

Compose by matrix multiplication, very important for performance!

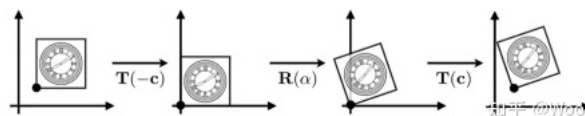
$$A_n (\dots A_2 (A_1 (x))) = A_n \cdot \dots \cdot A_2 \cdot A_1 \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Pre-multiply n matrices to obtain a single matrix representing combined transform

4. Decomposing Complex Transforms

How to rotate around a given point c ?

1. Translate center to origin
2. Rotate
3. Translate back



Matrix representation : $T_{(c)} \cdot R_{(\alpha)} \cdot T_{(-c)}$

五、3D Transformations

1. Definition

use homogeneous coordinate again

$$\text{3D point} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad \text{3D vector} = \begin{pmatrix} x \\ y \\ z \\ 0 \end{pmatrix}$$

In general, (x, y, z, w) ($w \neq 0$) is the 3D point:
 $(x/w, y/w, z/w)$

4×4 matrices for affine transformations :

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

2. Affine transformation's order

linear transform before translation

3. Scale

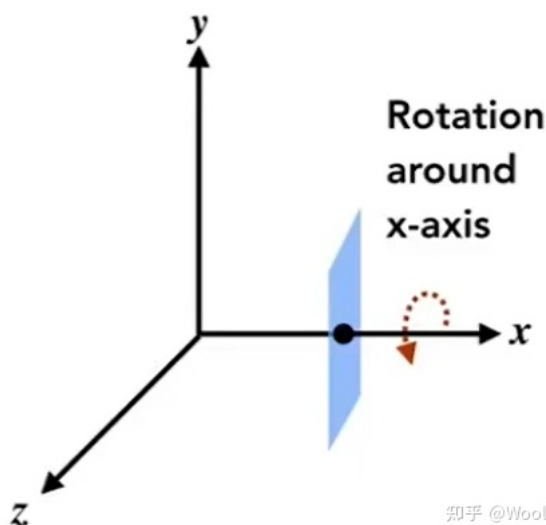
$$S_{(s_x, s_y, s_z)} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

4. Translation

$$T_{(t_x, t_y, t_z)} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

5. Rotation

(1) Rotation around x-, y-, or z-axis



知乎 @Wool

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

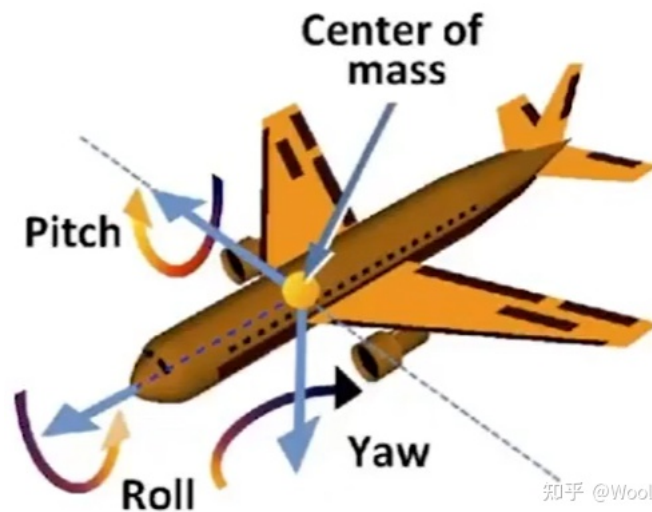
$$R_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \vec{y} = \vec{z} \times \vec{x} = -\vec{x} \times \vec{z}$$

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(2) Compose any 3D rotation from R_x, R_y, R_z ?

$$R_{xyz}(\alpha, \beta, \gamma) = R_x(\alpha) R_y(\beta) R_z(\gamma)$$

- So-called Euler angles
- Often used in flight simulators: roll, pitch, yaw



(3) Rodrigues's Rotation Formula

1) Rotation by angle α around axis n

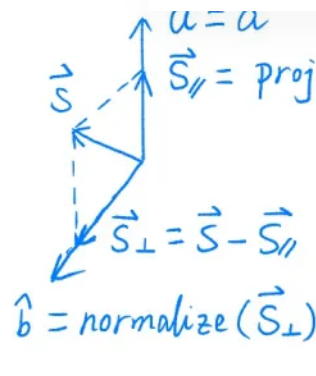
$$R(n, \alpha) = \cos(\alpha)I + (1 - \cos(\alpha))nn^T + \sin(\alpha) \underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_N$$

Note: the axis n is over the origin (default)

the way to use the axes not over the origin: translate the axis $\rightarrow$ rotate $\rightarrow$ translate back

2) proof

Step 1:

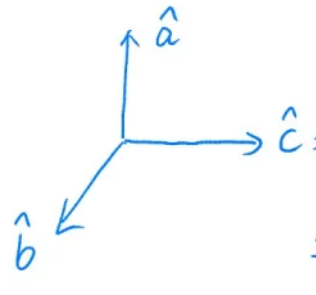


$$\vec{S}_{\parallel} = \text{proj}(\vec{S}, \hat{a}) = \hat{a}(\vec{S} \cdot \hat{a}) = a a^T \vec{S}$$

$$\vec{S}_{\perp} = \vec{S} - \vec{S}_{\parallel}$$

$$\hat{b} = \text{normalize}(\vec{S}_{\perp}) = \frac{\vec{S}_{\perp}}{\|\vec{S}_{\perp}\|}$$

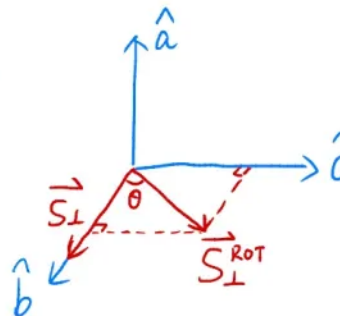
Step 2:



$$\hat{c} = \hat{a} \times \hat{b} = \frac{\hat{a} \times \vec{S}_{\perp}}{\|\vec{S}_{\perp}\|}$$

$$= \frac{\hat{a} \times \vec{S}}{\|\vec{S}_{\perp}\|}$$

Step 3:



$$\vec{S}_{\perp}^{\text{ROT}} = \|\vec{S}_{\perp}\| \cos \theta \hat{b} + \|\vec{S}_{\perp}\| \sin \theta \hat{c}$$

知乎 @Wool

supplement material

$$\begin{aligned} \vec{S}_{\text{rot}} &= R(n, \theta) \cdot \vec{S} = \vec{S}_{\parallel}^{\text{rot}} + \vec{S}_{\perp}^{\text{rot}} \\ &= \vec{S}_{\parallel} + \vec{S}_{\perp}^{\text{rot}} \\ &= \hat{n} \hat{n}^T \vec{S} + \left\| \vec{S}_{\perp} \right\| \cos(\theta) \hat{b} + \left\| \vec{S}_{\perp} \right\| \sin(\theta) \hat{c} \\ &= \hat{n} \hat{n}^T \vec{S} + \vec{S}_{\perp} \cos(\theta) + \sin(\theta) \hat{n} \times \vec{S} \\ &= \hat{n} \hat{n}^T \vec{S} + (\vec{S} - \vec{S}_{\parallel}) \cos(\theta) + \sin(\theta) \hat{n} \times \vec{S} \\ &= \cos(\theta) \vec{S} + (1 - \cos(\theta)) \hat{n} \hat{n}^T \vec{S} + \sin(\theta) \hat{n} \times \vec{S} \\ &= \cos(\theta) \vec{S} + (1 - \cos(\theta)) \hat{n} \hat{n}^T \vec{S} + \sin(\theta) \underbrace{\begin{bmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{bmatrix}}_N \vec{S} \end{aligned}$$

$$\therefore R(n, \alpha) = \cos(\alpha) I + (1 - \cos(\alpha)) n n^T + \sin(\alpha) \underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_N$$

 I : Identity Matrix and Inverses $\hat{n}$ how to become a matrix: Cross product

赞同

添加评论

分享

喜欢

收藏

申请转载

...

(4) Quaternions

Use: Find the interpolation between two rotation angles.

In a 2D plane, a graphic rotates 25° and 15° , add the two matrices and get the average, the result matrix is not the 20°

六、Viewing transformation

1. View / Camera Transformation

(1) What is view transformation?

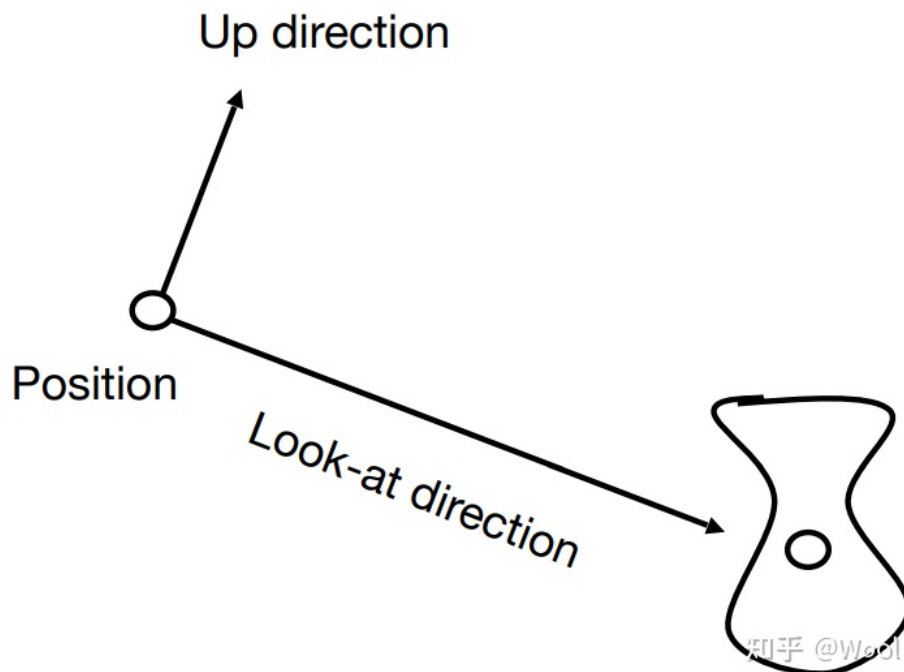
Think about how to take a photo

1. Find a good place and arrange people (**model** transformation)
2. Find a good "angle" to put the camera (**view** transformation)
3. Cheese! (**projection** transformation)

"MVP"

(2) How to perform view transformation?

1) Define the camera first



- Position $\vec{e}$
- Look-at / gaze direction $\hat{g}$
- Up direction $\hat{t}$

2) Key observation

If the camera and all objects move together, the "photo" will be the same.

$$(u + (1-u))^n \Rightarrow \text{展开式}$$

$$\begin{aligned} B_{4,0} &= \binom{4}{0} u^0 (1-u)^4 & B_{4,3} &= \binom{4}{3} u^3 (1-u) \\ B_{4,1} &= \binom{4}{1} u^1 (1-u)^3 & B_{4,4} &= \binom{4}{4} u^4 (1-u)^0 \\ B_{4,2} &= \binom{4}{2} u^2 (1-u)^2 \end{aligned}$$

要绘制曲线，也就是在 $[0, 1]$ 上取一些点，然后根据公式计算出我们所取点所对应的曲线上的点，再将点连接起来，就是我们需要的曲线。

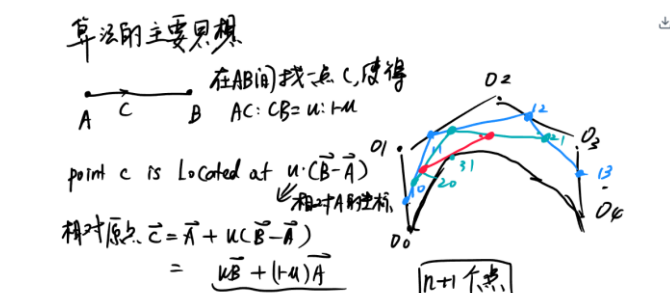
```
// 使用C++实现B-Spline曲线
for(i = 0; i <= num_cp-1; i++)
{
    x+=comb(num_cp-1,i)*pow(u,i)*pow((1-u),(num_cp-1-i))*p[i].x;
    y+=comb(num_cp-1,i)*pow(u,i)*pow((1-u),(num_cp-1-i))*p[i].y;
}
p1[num].x=x;
p1[num].y=y;
num++;
}
```

我们可以选择直接将系数计算出来后，再将 u 值代入公式计算对应点，但是这样总是需要计算阶乘，非常耗时，我们可以换一种思路。

首先我们将点进行编号

points	iter	Label
P_0	0	0
P_1	0	1
P_2	0	2
$\vdots$	$\vdots$	$\vdots$
P_n	0	n

再来介绍算法的思想：



2. B-Spline 曲线

实验二 基本图形生成算法

一、实验目的

- 1.掌握直线生成算法。
- 2.掌握圆和椭圆生成算法。

二、实验内容

- 1、 利用 OpenGL 绘图函数生成直线;
- 2、 利用 OpenGL 生成圆和椭圆;

三、实验环境

PC 微机

Windows 10 操作系统

CLion 集成开发环境

四、实验步骤

1. 根据 bresenham 算法, 绘制直线;
2. 根据中点画线算法, 绘制直线;
3. 根据 bresenham 画圆算法, 绘制圆;
4. 根据中点画圆算法, 绘制圆

五、实验结果

首先, 我自定义了 draw_pixel 函数

```
void draw_pixel(int ix, int iy, int value) {  
    glBegin( mode: GL_POINTS);  
    glVertex2i( x: ix, y: iy);  
    glEnd();  
}
```

1. 中点画线法

预处理步骤

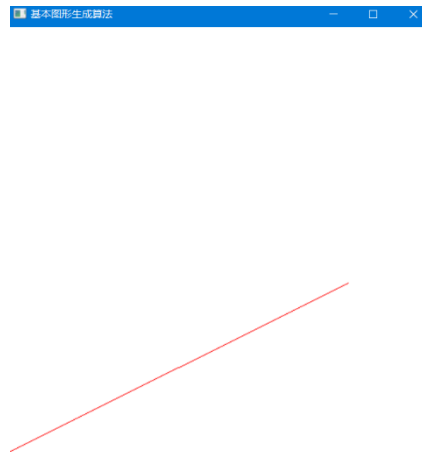
$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ len &= (\max(\Delta x, \Delta y))\end{aligned}$$

算法

```
while not in end point:
    if  $D_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + \Delta D$ 
```

```
void mid_point_draw_line(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;          delta x, delta y
    int dx = x2 - x1;
    int D = 2 * dy - dx; D_initial
    int delta_d = 2 * dy - 2 * dx; delta D
    int xk = x1, yk = y1;
    while (xk != x2 && yk != y2) { 迭代循环
        xk++;
        if (D < 0) {
            D += 2 * dy;
        } else {
            yk++;
            D += delta_d;
        }
        draw_pixel( ix: xk, iy: yk, value: BLACK);
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```

结果：



2. Bresenham 算法

前提条件

- 直线斜率 m 满足 $0 < m < 1$
- 在其他区域需要额外处理

输入

$(x_1, y_1), (x_n, y_n)$

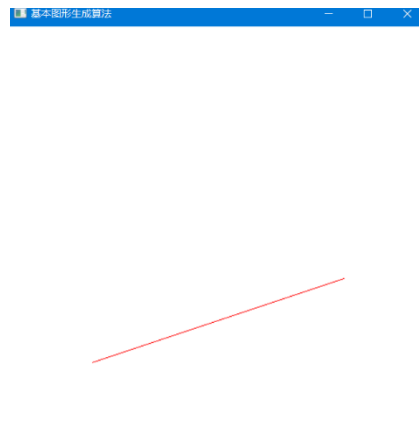
预处理步骤

$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ P_{initial} &= 2\Delta y - \Delta x\end{aligned}$$

算法

```
while not in end point:
    if  $P_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + 2 * (\Delta y - \Delta x)$ 
```

```
void bres(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;
    int dx = x2 - x1;
    int p = 2 * dy - dx;    p_initial
    int d1 = 2 * dy, d2 = 2 * (dy - dx);
    int xk = x1, yk = y1;
    int times = 0;
    while ((xk != x2 && yk != y2) || (times < dx)) {
        times++;
        xk++;
        if (p < 0) {
            p += d1;
        } else {
            p += d2;
            yk++;
        }
        draw_pixel(xk, yk, value: BLACK); 自定义绘图函数
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```



3. 中点画圆算法:

输入

$(x_0, y_0) R$

预处理步骤

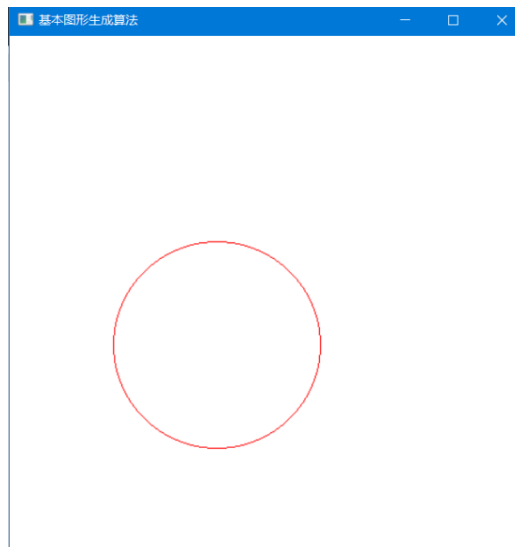
$$\begin{aligned}x_k &:= 0 \quad y_k = R \\ P_{\text{inital}} &= 1 - R\end{aligned}$$

算法

```
while  $x_c < y_c$ 
  if  $P_{\text{initial}} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 2 * x_k + 1$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 2 * y_k + 2 * x_k + 1$ 
```

```
void mid_point_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;          初始坐标
    int p = 1 - r;
    draw_whole_circle(x, y, xc, yc); drawCircle的作用是绘制 给定一点 (x,y),
    while (xc < yc) {             绘制该点在八个卦限的对应点
        xc++;
        if (p < 0) {
            p = p + 2 * xc + 1;
        } else {
            yc--;
            p = p - 2 * yc + 2 * xc + 1;
        }
        draw_whole_circle(x, y, xc, yc);
    }
}
```

```
void draw_whole_circle(int x, int y, int xc, int yc) {
    draw_pixel( ix: xc + x, iy: yc + y, value: 0);
    draw_pixel( ix: yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: yc + y, value: 0);
    std::cout << "(" << x << ", " << y << ")" << std::endl;
}
```



4. bresanham 画圆 算法

输入

(x_0, y_0) R

预处理步骤

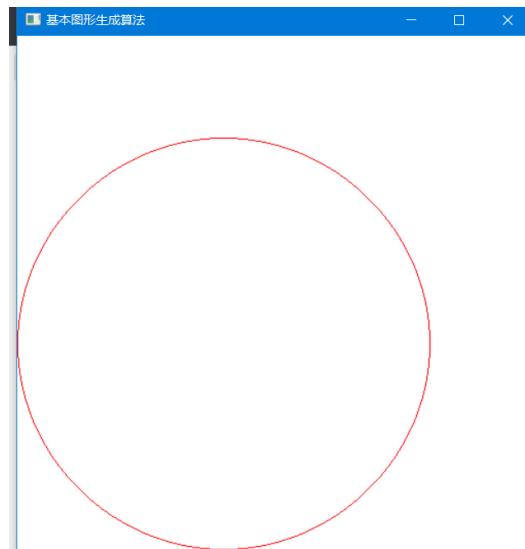
$$x_k := 0 \quad y_k = R$$

$$P_{initial} = 3 - 2R$$

算法

```
while  $x_c < y_c$ 
  if  $P_{initial} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 4 * x_k + 6$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 4 * y_k - 4 * x_k + 10$ 
```

```
void bresanham_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;
    int p = 3 - 2 * r;
    draw_whole_circle(x, y, xc, yc);
    while (xc < yc) {
        xc++;
        if (p < 0) {
            p = p + 4 * xc + 6;
        } else {
            yc--;
            p = p + 4 * (xc - yc) + 10;
        }
        std::cout << p << " ";
        draw_whole_circle(x, y, xc, yc);
    }
    std::cout << "finish" << " ";
}
```



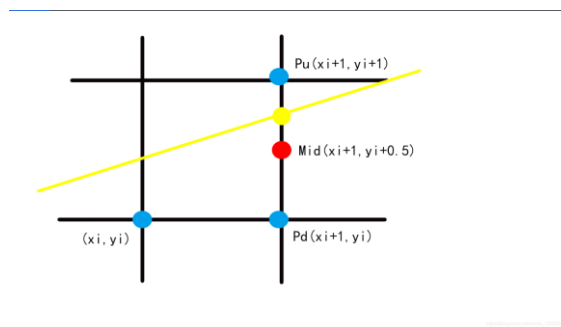
六、实验心得

1. 在本次实验中，使用中点画线法只能绘制斜率在 $0 \leq k \leq 1$ 范围内的直线，为了能够将其推广，我们先介绍中点画线算法的原理。

对于起始点 (x_0, y_0) 、结束点 (x_1, y_1) ，有 $A = y_0 - y_1$ 、 $B = x_1 - x_0$ 、 $C = (x_1 - x_0)(x_0 - y_1)$ 、 $k = (y_1 - y_0)/(x_1 - x_0)$ 。设函数 $F(x, y) = Ax + By + C$ ，则 $F(x, y)$ 将平面分为三部分：

- 1) $F(x, y) = 0$ ，点在直线上
- 2) $F(x, y) > 0$ ，点在直线的上方
- 3) $F(x, y) < 0$ ，点在直线的下方

在画线过程中，对于每个当前像素点 (x_i, y_i) ，每次在最大位移方向，即 x 方向上前进一步，在 y 方向上是否前进取决于前一格中点和直线的位置关系。



如上图所示，可判断中点 Mid 和直线的位置关系，来确定下一点的选取，令 d 为判式：

- 1) $d = F(x_i + 1, y_i + 0.5) = 0$ ， Mid 在直线上，可选取 Pu/Pd ，这里我们统一规定选取 Pd
- 2) $d = F(x_i + 1, y_i + 0.5) > 0$ ， Mid 在直线的上方，直线离 Pd 较近，选取 Pd ，则下一点 y 方向上不变
- 3) $d = F(x_i + 1, y_i + 0.5) < 0$ ， Mid 在直线的下方，直线离 Pu 较近，选取 Pu ，则下一点 y 方向上前进

利用增量思想，每下一步的 d 的判别可以基于本步 d 的正负情况，分本步 $d < 0$ 和 $d \geq 0$ 两种情况讨论。

1) $d < 0$: 本步选取 $P_u(x_i+1, y_i+1)$, 则下一步中点 $(x_i+2, y_i+1.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 1.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 1.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A + B \\ d_{new} &= d + A + B \end{aligned}$$

2) $d \geq 0$: 本步选取 $P_d(x_i+1, y_i)$, 则下一步中点 $(x_i+2, y_i+0.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 0.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 0.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A \\ d_{new} &= d + A \end{aligned}$$

在画线过程中每一点的情况都分析完了, 也易得结束画线的条件。接下来只要确定初始值就能顺利地画线了。对于起始点 (x_0, y_0) , 下一步中点为 $(x_0+1, y_0+0.5)$:

$$\begin{aligned} d_0 &= F(x_0 + 1, y_0 + 0.5) \\ d_0 &= A(x_0 + 1) + B(y_0 + 0.5) + C \\ d_0 &= [Ax_0 + By_0 + C] + A + 0.5B \\ d_0 &= A + 0.5B \end{aligned}$$

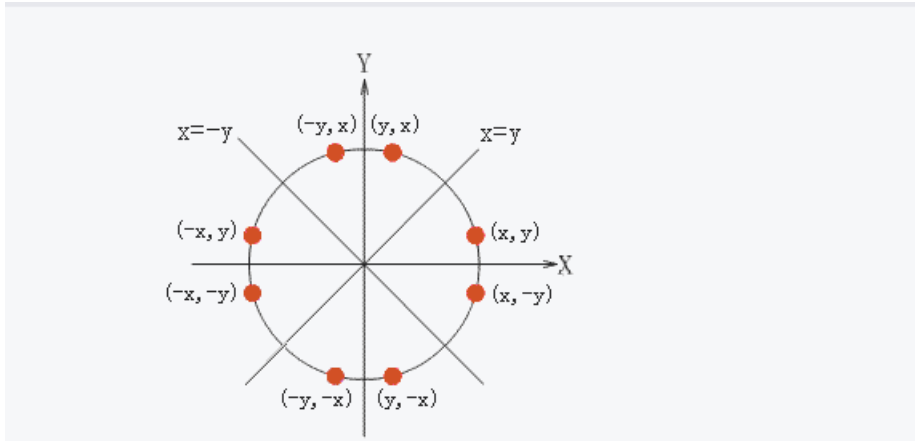
可以发现 d_0 的计算中涉及到了浮点数 0.5, 为了改进我们可以用 $2d_0$ 代替 d_0 , 因为后续计算中只考虑 d 的正负, 则扩大两倍不会对结果有所影响。此时的新 d_0 为:

$$\begin{aligned} d_0 &= 2F(x_0 + 1, y_0 + 0.5) \\ d_0 &= 2A(x_0 + 1) + 2B(y_0 + 0.5) + 2C \\ d_0 &= 2[Ax_0 + By_0 + C] + 2A + B \\ d_0 &= 2A + B \end{aligned}$$

进行推广

$k \in [0, 1]$	$k \in (1, +\infty)$	$k \in [-1, 0)$	$k \in (-\infty, -1)$
$d_0 = 2A + B$	$d_0 = A + 2B$	$d_0 = 2A - B$	$d_0 = A - 2B$
$d < 0$: $d += 2(A+B)$; $x += 1; y += 1$;	$d < 0$: $d += 2B$; $y += 1$;	$d < 0$: $d += 2A$; $x += 1$;	$d < 0$: $d += 2(A-B)$; $y -= 1; x += 1$;
$d \geq 0$: $d += 2A$; $x += 1$;	$d \geq 0$: $d += 2(A+B)$; $y += 1; x += 1$	$d \geq 0$: $d += 2(A-B)$; $x += 1; y -= 1$;	$d \geq 0$: $d -= 2B$; $y -= 1$;

2. 使用中点画圆法或者 bresenham 画圆法, 其实只能画出 $1/8$ 圆弧, 我们是利用圆的八分对称性, 画出整个圆来的。 $(x, -y)$ 、 $(-x, y)$ 、 $(-x, -y)$ 、 (y, x) 、 $(y, -x)$ 、 $(-y, x)$ 、 $(-y, -x)$



七、实验总结

这次实验主要让我们使用两种直线绘制算法（Bresenham 算法和中点画线算法）和两种圆绘制算法（Bresenham 画圆算法和中点画圆算法）绘制图形。

在实验中，我们首先学习了 Bresenham 算法和中点画线算法，然后分别使用这两种算法绘制直线。通过实验我们发现，Bresenham 算法具有更高的效率，能够更快地生成直线，而中点画线算法则更简单易懂。

接下来，我们学习了 Bresenham 画圆算法和中点画圆算法，并分别使用这两种算法绘制圆。实验结果表明，Bresenham 画圆算法和中点画圆算法都能生成圆形，但 Bresenham 画圆算法比中点画圆算法更快。

总的来说，这次实验让我更深入地了解了绘制图形的算法，并通过实际操作加深了对这些算法的理解。这对于我以后在图形学方面的学习和实践将会有很大的帮助。

八、完整代码

```
1 //
2 // Created by 1729 on 2022/11/24.
3 //
4 //A.15 Bresenham 画线算法
5
6 // for drawing a circle
7
8 #include<GL/glut.h>
9 #include <iostream>
10 #include <algorithm>
11
12 #define BLACK 1
13
14 void draw_pixel(int ix, int iy, int value) {
15     glBegin(GL_POINTS);
16     glVertex2i(ix, iy);
17     glEnd();
18 }
19
20 void draw_whole_circle(int x, int y, int xc, int yc) {
21     draw_pixel(xc + x, yc + y, 0);
```